

JavaScript Big O and Memory Concepts

Time Complexity • Space Complexity • Stack • Heap • References

English + தமிழ் | JavaScript | Beginner to Interview Ready

CONCEPT

REAL TIME PROGRAMS

INTERVIEW FOCUS

Prepared as structured study notes with runnable examples and revision questions.

How to study this guide

இந்த PDF-ஐ order-ஆ படிக்கவும்: Big O purpose → common complexities → code analysis → space complexity → JavaScript memory → primitive/reference → interview practice. Each code block can run directly in browser console or Node.js.

Learning outcome

- ஒரு program fast அல்லது slow ஆகும் pattern-ஐ Big O மூலம் explain பண்ண முடியும்.
- Loop, nested loop, binary search code-ன் time complexity கண்டுபிடிக்க முடியும்.
- Primitive copy, object reference, shallow copy, deep copy behavior புரியும்.
- Interview-ல் precise English answer + simple Tamil explanation சொல்ல முடியும்.

Quick map

Topic	Memory line	Typical example
$O(1)$	Direct-ah access	arr[index]
$O(\log n)$	ஒவ்வொரு step-லும் half	Binary Search
$O(n)$	ஒவ்வொண்ணா process	Single loop
$O(n \log n)$	Split + process	Merge Sort
$O(n^2)$	ஒவ்வொண்ணுக்கும் எல்லாம்	Nested loop

1 Big O Notation

Big O describes how an algorithm grows when the input size n grows. இது exact seconds சொல்லாது; input அதிகமானால் operations எவ்வளவு வேகமாக increase ஆகும் என்பதை சொல்லும் growth model.

Why developers need it

- Small test data-ல் எல்லா code-மும் fast போல தெரியலாம்; production data-ல் difference தெரியும்.
- Two solutions correct output தரினாலும் scalable solution-ஐ select செய்ய உதவும்.
- Interview-ல் solution சொன்ன பிறகு time and space complexity கேட்பது common.

Rules for beginners

Code pattern	Rule	Result
Fixed statements	Constants ignore	$O(1)$

Code pattern	Rule	Result
One loop up to n	Keep n	O(n)
Consecutive loops	Add, then keep dominant	O(n)
Nested full loops	Multiply	O(n ²)
Input halves each step	Logarithmic	O(log n)

2 O 1 Constant Time

The number of operations stays roughly constant even when input grows. Input 5 இருந்தாலும் 5 lakh இருந்தாலும் direct index access one main step.

```
const students = ['Asha', 'Bala', 'Charan', 'Divya'];
console.log(students[2]); // Charan
```

Real time example Student ID lookup

```
const studentById = {
  ST101: { name: 'Asha', mark: 88 },
  ST102: { name: 'Bala', mark: 74 }
};

console.log(studentById.ST102); // direct key lookup
// Average time: O(1)
```

Use case: dashboard-ல் student ID தெரிந்தால் list முழுக்க search செய்யாமல் object or Map key மூலம் direct record பெறலாம்.

3 O n Linear Time

Every item may need to be visited once. n doubles ஆனால் work roughly double ஆகும். Search target last-ல் இருந்தால் அல்லது இல்லையெனில் full list scan ஆகும்.

```
function findStudent(names, target) {
  for (const name of names) {
    if (name === target) return true;
  }
  return false;
}

console.log(findStudent(['Asha', 'Bala', 'Charan'], 'Charan'));
// Worst-case time: O(n), space: O(1)
```

Real time example Total cart amount

```
function cartTotal(items) {
  let total = 0;
  for (const item of items) total += item.price * item.qty;
  return total;
}

console.log(cartTotal([
  { price: 500, qty: 2 },
  { price: 250, qty: 1 }
])); // 1250
```

4 $O(n^2)$ Quadratic Time

A full loop inside another full loop usually gives $n \times n$ operations. 100 records என்றால் 10,000 comparisons வரலாம். Small data-க்கு okay; large data-க்கு costly.

```
const students = ['Asha', 'Bala', 'Charan'];
for (let i = 0; i < students.length; i++) {
  for (let j = i + 1; j < students.length; j++) {
    console.log(students[i], students[j]);
  }
}
// Pair generation:  $O(n^2)$ 
```

Real time example Duplicate detection slow version

```
function hasDuplicateSlow(values) {
  for (let i = 0; i < values.length; i++) {
    for (let j = i + 1; j < values.length; j++) {
      if (values[i] === values[j]) return true;
    }
  }
  return false;
}
```

5 $O(\log n)$ Logarithmic Time

Each step removes half of the remaining search area. இது sorted data-க்கு Binary Search-ல் common. 1,024 values இருந்தாலும் around 10 halving steps போதும்.

```
function binarySearch(sorted, target) {
  let left = 0, right = sorted.length - 1;
  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (sorted[mid] === target) return mid;
    if (sorted[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1;
}

console.log(binarySearch([10,20,30,40,50,60], 50)); // 4
// Time:  $O(\log n)$ , extra space:  $O(1)$ 
```

Important: Binary Search பயன்படுத்த input sorted ஆக இருக்க வேண்டும். Unsorted list-ல் directly use செய்யக்கூடாது.

6 $O(n \log n)$ Efficient sorting growth

Divide-and-conquer algorithms often split data logarithmically and process n items across levels. Merge Sort average/worst time $O(n \log n)$. JavaScript `Array.prototype.sort` complexity engine-dependent; interview-ல் algorithm specified இருந்தால் அதற்கான complexity சொல்லவும்.

```
function merge(left, right) {
  const out = []; let i = 0, j = 0;
  while (i < left.length && j < right.length)
    out.push(left[i] <= right[j] ? left[i++] : right[j++]);
  return out.concat(left.slice(i), right.slice(j));
}
function mergeSort(a) {
  if (a.length <= 1) return a;
  const mid = Math.floor(a.length / 2);
  return merge(mergeSort(a.slice(0, mid)), mergeSort(a.slice(mid)));
}
console.log(mergeSort([8,3,5,1,9]));
```

7 Comparing common complexities

Complexity	n 10	n 100	Meaning
$O(1)$	1	1	Fixed work
$O(\log n)$	~3	~7	Repeated halving
$O(n)$	10	100	One pass
$O(n \log n)$	~33	~664	Efficient sorting pattern
$O(n^2)$	100	10,000	All pairs / nested work

These operation counts are simplified growth illustrations, not real execution-time guarantees. Hardware, JavaScript engine, I/O, constants, and data shape also affect runtime.

8 How to calculate Big O from code

Example A Consecutive loops

```
for (const x of a) console.log(x); //  $O(n)$ 
for (const x of a) console.log(x); //  $O(n)$ 
//  $O(n + n) = O(2n) = O(n)$ 
```

Example B Different inputs

```
for (const x of users) console.log(x); // O(u)
for (const y of products) console.log(y); // O(p)
// Total: O(u + p), not automatically O(n)
```

Example C Nested loops with different inputs

```
for (const user of users) {
  for (const product of products) console.log(user, product);
}
// O(u × p)
```

Example D Early return

```
function contains(a, target) {
  for (const x of a) if (x === target) return true;
  return false;
}
// Best: O(1), Worst: O(n)
```

9 Space Complexity

Space complexity measures extra memory used as input grows. Input storage-ஐ exclude செய்து auxiliary space சொல்லுவது common. Always clarify what you count.

O 1 extra space

```
function maxValue(a) {
  let max = -Infinity;
  for (const x of a) if (x > max) max = x;
  return max;
}
// Time O(n), extra space O(1)
```

O n extra space

```
function doubleValues(a) {
  const result = [];
  for (const x of a) result.push(x * 2);
  return result;
}
// Time O(n), extra space O(n)
```

10 JavaScript Memory Basics

Beginner mental model: primitive values behave like independent copied values; objects, arrays, and functions are accessed through reference values. “Stack and heap” is a useful model, but exact storage is decided by the JavaScript engine and can be optimized.

Area	Beginner mental model	Important note
Call stack	Function calls and local execution frames	LIFO execution structure
Heap	Dynamically managed objects and arrays	Garbage collector manages reachability
Reference value	A value that identifies an object	JS does not expose raw memory addresses

11 Primitive Values

Primitive types: string, number, bigint, boolean, undefined, symbol, and null. Assignment copies the primitive value, so updating the second variable does not update the first.

```
let originalMark = 80;
let copiedMark = originalMark;
copiedMark = 95;
console.log(originalMark); // 80
console.log(copiedMark);   // 95
```

12 Reference Values

Objects and arrays can be shared by multiple variables. Assignment copies the reference value, not a new independent object. So a mutation through one variable is visible through the other.

```
const student1 = { name: 'Asha', mark: 80 };
const student2 = student1;
student2.mark = 95;
console.log(student1.mark); // 95
console.log(student1 === student2); // true
```

Reference diagram

Variable	Points to	Observed mark
student1	same student object	95
student2	same student object	95

13 Functions and object mutation

JavaScript is pass-by-value. When an object is passed, the copied value is a reference to that object. The function can mutate the shared object, but reassigning the local parameter does not replace the caller variable.

```
function updateName(person) {
  person.name = 'Kavin';      // shared object mutation
}
function replaceLocally(person) {
  person = { name: 'New' };   // local parameter reassignment
}
const user = { name: 'Arun' };
updateName(user);
replaceLocally(user);
console.log(user.name); // Kavin
```

14 Shallow Copy and Deep Copy

Shallow copy

```
const user1 = { name: 'Asha', address: { city: 'Chennai' } };
const user2 = { ...user1 };
user2.name = 'Bala';           // independent top level
user2.address.city = 'Madurai'; // nested object is shared
console.log(user1.name);       // Asha
console.log(user1.address.city); // Madurai
```

Deep copy for supported data

```
const user3 = structuredClone(user1);
user3.address.city = 'Coimbatore';
console.log(user1.address.city); // Madurai
```

structuredClone supports many built-in data types but does not clone functions and some special objects. JSON.parse(JSON.stringify(...)) can lose undefined, Date behavior, BigInt, and other data; do not present it as a universal deep-copy solution.

15 Real Time Optimization Programs

Program 1 Duplicate student ID O n

```
function hasDuplicateId(ids) {
  const seen = new Set();
  for (const id of ids) {
    if (seen.has(id)) return true;
    seen.add(id);
  }
  return false;
}
// Time O(n); extra space O(n)
```

Nested loop $O(n^2)$ -ஐ Set use செய்து average $O(n)$ solution-ஆ மாற்றுகிறோம். Extra memory $O(n)$ என்ற trade-off இருக்கு.

Program 2 Attendance frequency $O(n)$

```
function attendanceCount(records) {
  const count = {};
  for (const name of records) count[name] = (count[name] ?? 0) + 1;
  return count;
}
console.log(attendanceCount(['Asha', 'Bala', 'Asha']));
// { Asha: 2, Bala: 1 }
```

Program 3 Two Sum using Map

```
function twoSum(nums, target) {
  const indexByValue = new Map();
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (indexByValue.has(need)) return [indexByValue.get(need), i];
    indexByValue.set(nums[i], i);
  }
  return [];
}
console.log(twoSum([2,7,11,15], 9)); // [0,1]
// Average time  $O(n)$ , space  $O(n)$ 
```

Program 4 First non repeating character

```
function firstUnique(text) {
  const freq = new Map();
  for (const ch of text) freq.set(ch, (freq.get(ch) ?? 0) + 1);
  for (const ch of text) if (freq.get(ch) === 1) return ch;
  return null;
}
console.log(firstUnique('swiss')); // w
```

16 Interview Questions and Answers

1 What is Big O notation?

It describes the upper-bound growth rate of time or space as input size grows. Exact runtime அல்ல; scalability pattern.

2 What is the difference between $O(n)$ and $O(n^2)$?

$O(n)$ usually grows linearly; $O(n^2)$ grows quadratically, often due to nested full loops.

3 Is $O(1)$ always faster than $O(n)$?

Asymptotically it scales better, but constants and real workloads matter for small inputs.

4 What are best and worst cases?

They describe minimum and maximum work for an input size. Linear search best $O(1)$, worst $O(n)$.

5 What is auxiliary space?

Extra memory used by the algorithm excluding the input itself.

6 Are JavaScript objects passed by reference?

Precise answer: JavaScript passes values. For objects, the passed value is a reference to the object.

7 Why are two equal-looking objects not equal with triple equals?

Object equality compares identity/reference, not deep property equality.

8 What is shallow copy?

A new outer object is created, but nested object references can still be shared.

9 When can binary search be used?

When the searchable data is sorted under the same comparison rule.

10 How can Set improve duplicate detection?

It replaces repeated pair comparisons with average constant-time membership checks, giving average $O(n)$ time.

17 Practice Tasks

1. Find the minimum number in an array and write time and space complexity.
2. Implement linear search and mention best/worst cases.
3. Implement binary search for a sorted marks list.
4. Find duplicates first using nested loops, then optimize using Set.
5. Count words using Map or object.
6. Explain the complexity of two consecutive loops.
7. Explain the complexity of two nested loops with different input arrays.
8. Create a shallow-copy example with a nested address.
9. Use structuredClone and prove the nested copy is independent.
10. Solve Two Sum using Map and dry-run it on paper.

Final revision sheet

Remember	Meaning
$O(1)$	Direct access
$O(\log n)$	Half each step
$O(n)$	Visit each item
$O(n \log n)$	Efficient divide and process
$O(n^2)$	All-pairs work
Primitive assignment	Independent value copy
Object assignment	Reference value copy
Spread	Shallow copy
structuredClone	Deep clone for supported values

Interview closing line: "I will first make the solution correct, then state the time and auxiliary space complexity, and finally explain any memory-versus-speed trade-off."